代码仓库 https://github.com/Kallen-Dis/rengongzhinengzuoye/tree/experiment2

实验二 搜索算法实验报告

实验日期 2026年10月16日

一 实验设置与路径搜索

本实验实现 A*、Minimax、Alpha-beta 及宝藏分配。Python 3.12.14，Windows 11；算法仅依赖标准库。每组预热 3 次，计时 51 次取中位数，单位 ms。包含统计、还原和排序，不含打印及文件写入；博弈不含距离预计算，后者单独测量。

A* 状态为坐标，g 为已付费用，h 为剩余费用下界，f=g+h。堆取最小 f；若 new_g 更小，更新 g_score 与前驱 came_from 并入堆。出堆时跳过 g 已过期的记录，抵达目标后沿前驱倒推路径。f 相同按坐标排序，保证结果可复现。

题目一 最短路径与对照

实际路径：(0,0)→(0,1)→(0,2)→(1,2)→(1,3)→(1,4)→(2,4)→(3,4)。8 个坐标对应 7 步。起点曼哈顿下界也是 7，可行路径达到下界，故最短。四方向单位费用下，曼哈顿距离可采纳且一致；h=0 等价于 Dijkstra。

题目	启发函数	步数或代价	扩展节点	时间 ms
题目一	h=0	7	15	0.0231
题目一	曼哈顿	7	13	0.0227
题目二	h=0	7	14	0.0431
题目二	修正曼哈顿	7	10	0.0363

题目一扩展从 15 降至 13，减少 13.3%；题目二从 14 降至 10，减少 28.6%。扩展计数仅包含真正枚举邻居的节点，不含终点；有效出堆计数另包含终点。

题目二 费用累计与启发函数修正

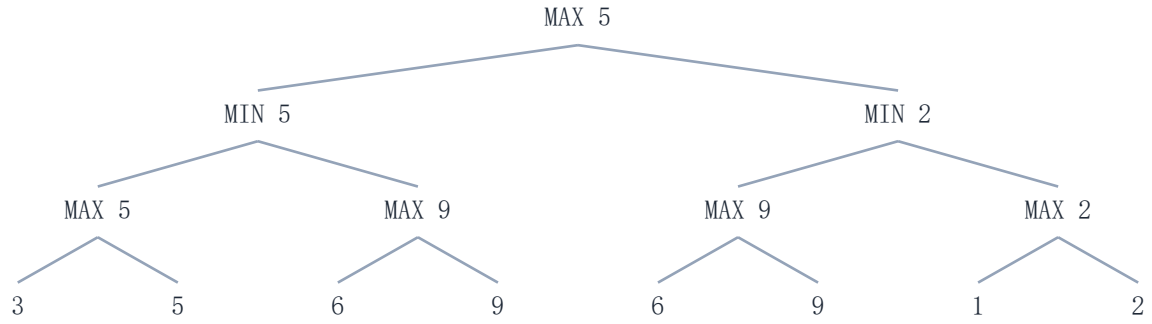
路径：(0,0)→(1,0)→(2,0)→(3,0)→(3,1)→(3,2)→(4,2)→(4,3)→(4,4)。进入费用依次为 1、1、1、1、1、1、1、0；累计为 1、2、3、4、5、6、7、7。出发不支付起点费，进入 E 免费，所以 8 步只花 7。

紧邻 E 时，普通曼哈顿估计 1，真实剩余费用 0，故会高估。令 d 为曼哈顿距离，m 为除免费目标外的最小通行费用，修正 h=max(d-1,0)×m：至少 d 步中，只有末步可免费，其余每步至少 m。相邻普通格的 h 下降不超过 m，进入目标前 h=0，因此也满足一致性。若其他格有零费用，m=0，安全退化为 Dijkstra。

网格中 E=O(V)，一致启发下堆实现时间 O((V+E)log V)，空间 O(V+E)。最小费用不等于最少步数；原样例正确也不能代替对启发性质的分析。

二 题目三 极大极小搜索与剪枝

叶子保存 MAX 视角的分数；非叶子保存 children。递归遇到叶子直接返回分数，MAX 层取最大，MIN 层取最小，根是 MAX。按 PDF 第 10 至 12 页完整输入展开如下，重复的 (6,9) 分支分别访问。



图一 博弈树的分数回传。默认从左到右访问，逆序仅用于对照。

底层 MAX 依次返回 5、9、9、2；两个 MIN 返回 min(5,9)=5 与 min(9,2)=2；根返回 max(5,2)=5。全部分数始终从 MAX 角度解释，MIN 不是将叶子取负，而是选择较小的结果。

剪枝依据与一次具体剪枝

alpha 为 MAX 已建立的下界，beta 为 MIN 已建立的上界。alpha≥beta 时，当前节点剩余分支无法改善祖先的决定，可以停止。左 MIN 得到 5 后令 beta=5；接着它的第二个 MAX 先看到叶子 6，令 alpha=6，便跳过后面的 9。

算法	顺序	值	访问	叶子	剪枝	时间 ms
Minimax	左到右	5	15	8	0	0.0039
Minimax	逐层逆序	5	15	8	0	0.0050
Alpha-beta	左到右	5	14	7	1	0.0054
Alpha-beta	逐层逆序	5	14	7	1	0.0068

两种顺序的 Alpha-beta 都仅省一个叶子；本树逆序没有改变总访问量。访问顺序能影响剪枝，但不是每棵树换序都会有差异。实测 Alpha-beta 比完整 Minimax 更慢，说明小树上边界维护开销可能超过少访问一个叶子的收益。

计数口径与复杂度

visited 是递归进入次数，含根与终止节点；leaves 是终止评价次数；cutoffs 只统计确实跳过剩余分支的剪枝。pruned_children 仅计跳过的直接子分支，不冒充其全部后代。A* generated 计入堆记录，可能含改进后的重复坐标；max_frontier 同样不等于去重坐标数。

分支因子 b、深度 H 下，Minimax 最坏 O(b^H)；Alpha-beta 最坏仍为 O(b^H)，理想顺序可接近 O(b^(H/2))。两者深度优先递归栈 O(H)，不计输入树存储。

三 题目四 状态建模与深度决策

宝藏价值 T1=9、T2=6、T3=7、T4=7。A、B、T 格的进入费用均为 1，不能当免费 E。先对六个关键点逐方向调用 A* 共 30 次，得到下表；对角为 0，行是出发点，列是目的地。

出发到达	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

状态记录双方位置、剩余宝藏、双方累计收益和当前玩家。选择 Ti 后，加上 Gain=Vi-d(p,Ti)，更新当前玩家位置、移除该宝藏并换人；新建不可变状态避免分支污染。经过其他宝藏不自动收集，已取宝藏仍可通行，另一玩家不阻挡，所以全局复用距离矩阵。

U=Score_A-Score_B；A 取最大，B 取最小。深度为 0 或宝藏取完时返回当前 U。H=2 为 A、B 各一次；H=4 为 A、B、A、B 四次。固定首步后对后续应对完整搜索，结果如下。

A 的首步	H=2 最优应对后的 U	H=4 最优应对后的 U
T1	-4	-5
T2	-1	1
T3	-2	3
T4	-4	-7

H=2 选择 T2：A 得 6-4=2，B 取 T3 得 7-4=3，U=-1。H=4 选择 T3，下面列出默认顺序得到的一条双方最优行动序列。

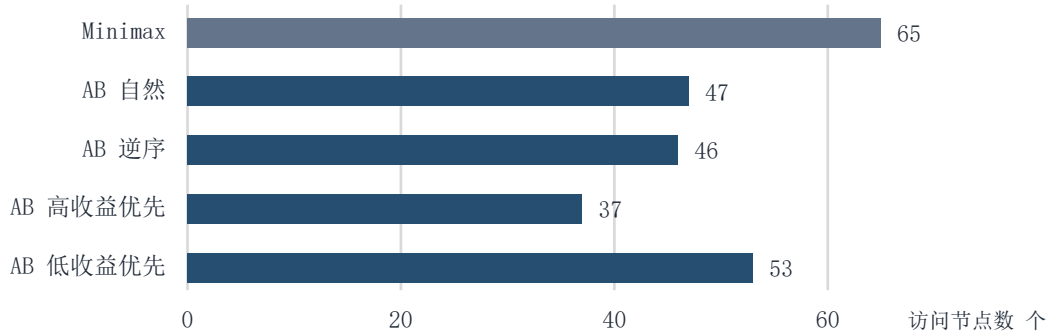
玩家	移动	费用	本次收益	累计 U
A	A → T3	8	-1	-1
B	B → T1	8	1	-2
A	T3 → T2	6	0	-2
B	T1 → T4	12	-5	3

A 首取 T3 虽立即亏 1，却拿走 B 容易获利的目标。在此序列中，B 最后花 12 取 T4，收益 -5；最终 A=-1、B=-4，U=3。负收益合法且不能跳过回合。最优应对可能不唯一，单条有利序列不能替代对所有应对的比较。

四 综合题性能与访问顺序

比较自然、逆序、即时高收益优先和即时低收益优先。双方都按自己的即时收益排序；只调整访问先后，不删除动作。高即时收益不保证完整深度最优，排序的作用是较早建立强的 alpha 或 beta。

深度	算法及顺序	访问	叶子	剪枝	时间 ms
2	MM 自然	17	12	0	0.0298
2	AB 自然	13	8	2	0.0240
2	AB 逆序	16	11	1	0.0322
2	AB 高收益优先	11	6	3	0.0265
2	AB 低收益优先	16	11	1	0.0374
4	MM 自然	65	24	0	0.1099
4	AB 自然	47	16	5	0.0847
4	AB 逆序	46	16	4	0.0908
4	AB 高收益优先	37	12	6	0.0815
4	AB 低收益优先	53	19	2	0.1153



图二 给定宝藏地图 H=4 的访问节点数，包含根与终止节点。MM 为 Minimax，AB 为 Alpha-beta。

完整树为 1+4+12+24+24=65 次访问。默认剪枝为 47，减少 27.7%；高收益优先为 37，减少 43.1%；低收益优先为 53。所有顺序的最优首步和值一致。剪枝次数不能代表省下的全部节点量，且排序本身有开销；耗时受系统调度影响，应结合节点数判断。

预计算与缓存的实际效果

距离矩阵的 h=0 总扩展 652 次，耗时 1.6998 ms；收费目标曼哈顿总扩展 287 次，耗时 1.0623 ms。收费目标使用 h=d×最小通行费用；关键点进入费相同使本题距离恰好对称，但程序仍逐方向求解。

精确值缓存的键含完整状态与剩余深度，H=2、4 均命中 0 次，访问仍为 17、65，耗时 0.0357、0.1371 ms；没有观察到收益。缓存需要重复状态才能有效。Alpha-beta 截断结果可能仅为界，本实现未将其误存为精确值。

五 自设实验 正确性验证与结论

地图	最短步数	h=0 扩展	曼哈顿扩展	减少
空旷 25×25	24	456	24	94.7%
绕墙 25×25	72	600	336	44.0%
随机 25×25	48	497	399	19.7%

空旷图从 (12,0) 到 (12,24)，启发精确指向同行，扩展从 456 降到 24。绕墙图从 (0,0) 到 (0,24)，第 12 列只有底部缺口，真实距离 72 远大于下界 24，所以仍需较多探索。随机图种子 2026、障碍概率 0.22，固定顶行及右列可通行；这不是无条件随机分布，不能把单图收益推广到所有地图。

边界输入	结果	验证内容
S 紧邻 E	代价 0	免费终点的启发下界
S 9 E 下方全为 1	4 步 代价 3	比直接 2 步代价 9 更便宜
S X E	返回 -1	不可达处理
S 0 0 E	代价 0	m=0 退化后仍正确

独立验证与复现

10 个 unittest 方法全部通过：用独立 BFS 验证迷宫；独立 Dijkstra 验证加权样例、6×6 关键点距离及 60 张种子 1431 的随机图；其中一半仅正费用、一半允许零费用，并逐点检查可采纳性、逐边检查一致性。完整 Minimax 核对给定树及 50 棵种子 2026 的随机树；枚举全部不重复行动排列并逐层归约，独立核对 H=2、H=4。另测起终点相同、零分叶子、负收益、重复收集和状态隔离。

在实验二目录执行 python run_experiments.py 生成 42 组数据；执行 python -m unittest discover -s code -p "test_*.py" -v 运行验证；执行 python build_report.py 重建不超过 5 页的报告。完整数据为 results/experiment_results.json 与 metrics.csv，详细说明见 README。

结论与局限

A* 的安全下界必须反映费用规则，尤其是免费终点和零费用地形。Alpha-beta 保持 Minimax 的值，实际收益受访问顺序和计算开销影响。宝藏题中，完整四回合的位置变化与竞争令最优首步从 T2 变为 T3，说明短期即时收益不足以决定长期策略。

距离预计算依赖地图和通行规则不变；若玩家阻路或取宝藏改变地形，应重新评估复用条件。当前测量只覆盖给定输入与有限自设场景，计时受机器影响；扩大地图、增加宝藏和多随机分布对照，可进一步检验排序、缓存与复杂度瓶颈。